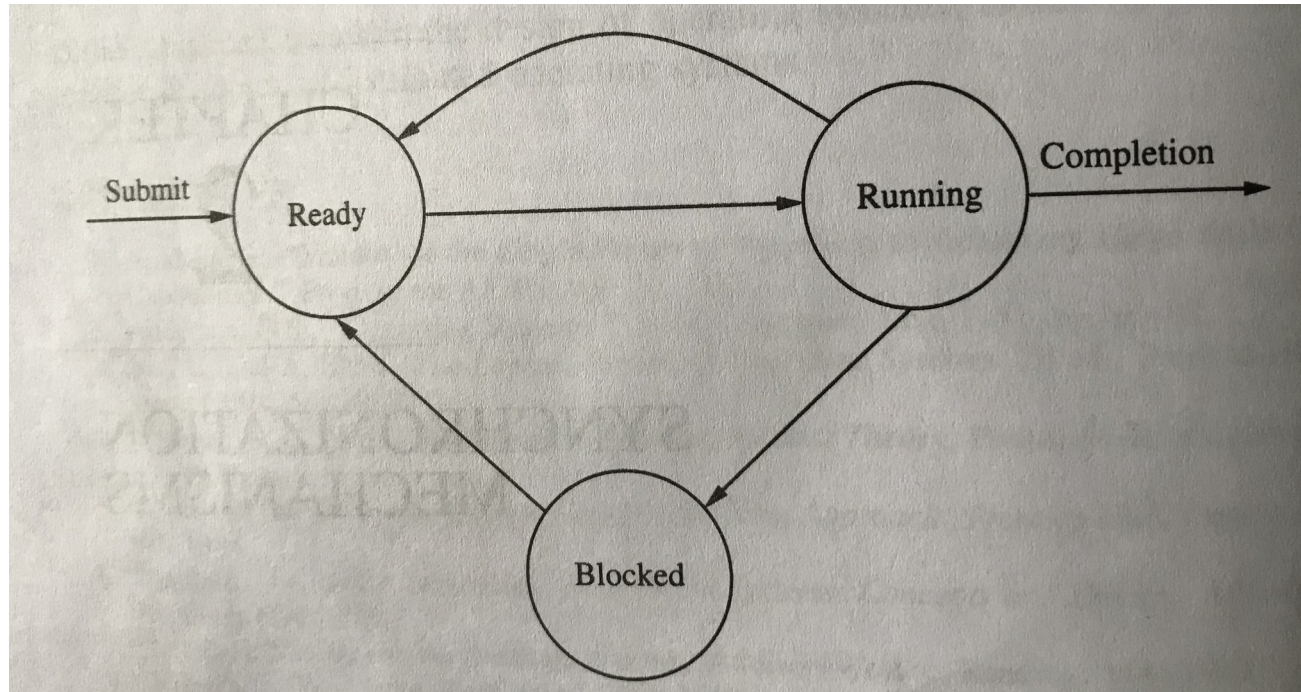


Concept of process

A process is a program whose execution has started but is not yet complete(a program in execution)

A process can be in any of the following three basic states

1. Running : The processor is executing the instructions of the corresponding process
2. The process is ready to be executed , but the processor is not available for execution of this process
3. Blocked:- The process is waiting for an event to occur. examples are I/O operation waiting to be completed



A running process gets blocked because a requested resource is not available or can become ready because the CPU decided to execute another process.

A blocked process becomes ready when the needed resource becomes available to it.

A ready process starts running when the CPU becomes available to it.

A data structure commonly referred to as The Process Control Block(PCB), stores complete information about a process such as Id, process state, priority, privileges, virtual memory address space transition maps etc.

Concurrent Process

Two process are concurrent if their execution can overlap in time; that is execution of second process starts before the first process completes. In multiprocessor systems, since the CPU can simultaneously execute different process, the concept of concurrency is concrete and easy to visualize

Two mechanism used in concurrent processes is

- 1. shared variables:- The process access(read or write) a common

Variable or common data.

2. Message passing:- The process exchange information with each other by sending and receiving messages.

Threads

To handle situations where creating , maintaining , and switching between process occur frequently threads or lightweight process have been proposed.

A single thread execute portion of program cooperating with other threads concurrently executing with the same address space.

Each Thread makes use of separate program counter and stack of activation records and a control block.

The control block contains information necessary for thread management , such as putting a thread in to ready list and for synchronizing with other threads.

The critical section problem

A critical section is a code segment in a process in which a shared resource is accessed. A solution to the problem of mutual exclusion must satisfy the following requirements:-

1. Only one process can execute its critical section at any one time.
2. When no process is executing in its critical section, any process that requests entry to its critical section must be permitted to enter without delay.
3. When two or more process compete to enter their respective critical sections, the section can not be postponed indefinitely.
4. No process can prevent any other process from entering its critical section indefinitely.

Mechanism for mutual Exclusion

Busy waiting:-Continuously test the value of a status variable to find if the shared resource is free

Disabling interrupts:- When a process disables the interrupts before entering the critical section and enables the interrupts immediately after exiting the critical section.

Test and set instruction:- When this instruction is executed , a specified memory location is checked for a particular value; if they match , the memory location 's contents are altered.

Semaphores

A semaphore is a high level construct used to synchronize concurrent process. A semaphore S is an integer variable on which process can perform two individual operations. $P(S)$ and $V(s)$ Each semaphore has a queue associated with it , where process that are blocked on that semaphore wait. The P and V operation are defined as

$P(s)$: if $S \geq 1$ Then $S = S - 1$ else block the process 0: the semaphore queue:

$V(S)$: If some process are blocked on the semaphore S Then unlock a process

Else $S := S + 1$;

When V(s) operation is performed, a block process is picked up for execution

Two types of semaphore

A binary semaphore(the initial value is 1) are used to create mutual exclusion ,because at any given time only one process can get past the P operation.

A resource counting semaphore(The initial value is normally more than 1) are primarily used to synchronize access to shared resource by several concurrent process.

Other synchronization problem

- The dining philosophers problem

Two phases : Thinking and eating

- The producer consumer problem

A set of producer process supplies messages to a set of consumer process.

- The Readers- Writers problem

The shared resource is a file that is accessed by both the reader and writer process.

- Language mechanism for synchronization

Monitors

Monitors are abstract data types for defining shared objects(resources) and for scheduling access to the objects in multiprogramming environments. A monitor consists of procedures, the shared objects, and administrative data.

- Only one process can be active within the monitor at a time.
- Procedures of a monitor can only access data local to the monitor: they cannot access an outside variable.
- The variables or data local to monitor cannot be directly accessed from outside the monitor.
- Two operations are wait and Signal

< Monitor-name>:monitor begin
Declaration of data local to the monitor.

:
:

procedure < Name> (< formal parameters>);

begin

procedure body

end;

Declaration of other procedures

:
:

begin

Initialization of local data of the monitor

end;

end;

FIGURE 2.4

The structure of a n

Serializers

Hewitt and Atkinson proposed serializers as a synchronization mechanism to overcome some of the deficiencies of monitors.

- Serializers allows concurrency inside and thus shared resource can be encapsulated in a serializer.
- Serializers replace explicit signaling required by monitors with automatic signaling.
- Procedures of a serializer may have hollow regions where in multiple process can be concurrently active when a process enters a hollow region, it releases the possession of the serializer and consequently , some other process can gain the possession of serializer.
- Any number of processes can be active in hollow region

- A hollow region in a procedure is specified by a join-crowd operation that allows processes to access the resource while releasing the serializer , thereby allowing concurrency.
- The syntax of the join-crowd command is
join-crowd(<crowd>) then <body> end
- Instead of condition variables , a serializer has queue variables.
- An enqueue operation , along with the condition the process is waiting for, provides a delaying or blocking facility. The syntax of enqueue is
enqueue(<priority>, <queue name>)until (≤<condition>)
-

Path expressions

- The concept of path expressions was proposed by Campbell and Habermann.
- A path expression restricts the set of admissible execution of the operations on the shared resource so that no incorrect state is ever reached and it indicates the order in which operations on a shared resource can be interleaved.
- A path expression has the following form

path S end;

where s denotes possible execution histories . It is an expression whose variables are on the resource and whose operations are:

- 1. Sequencing(;) it defines sequencing order among operations

For example path open; read; close; end means an open performed first followed by a read and a close in that order.

- 2. selection(+). It signifies that only one of the operations connected by + operator can be executed at a time. For example path read+write end means that only read or only write can be executed at a give time.
- 3. Concurrency({}). It signifies that any number of instances of the operation delimited by { and} can be in execution time. For example path{read} end means that any number of read operations can be executed concurrently. The path expression path write: {read} end allows either several read operations or a single write operation to be executed at any time.